VIEWMODEL

SENERYO – 1

SENERYO – 2

DTO(DATA TRANSFER OBJECT) NEDIR ?

DTO ILE VIEWMODEL ARASINDAKI FARKLAR

TEMEL AMAÇ

SENERYO - 1

SENERYO - 2

SÖZLEŞME/KONTRAR MANTIĞI NEDIR ?

VIEWMODEL'LAR DA VALIDATION DURUMLARI

VIEWMODEL'I ENTITY'E DÖNÜŞTÜRME

Manuel Dönüştürme

Implicit Operator Overload ile Dönüştürme

Explicit Operator Overload ile Dönüştürme

Reflection ile Dönüştürme

AutoMapper Kütüphanesi ile Dönüştürme

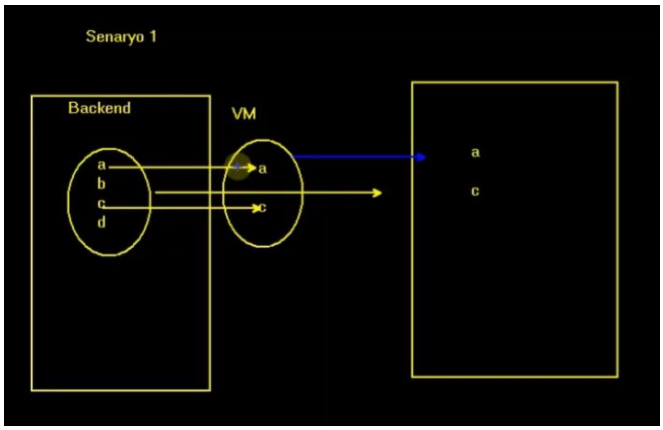
ViewModel

ViewModel, temelde iki farklı senaryoya karşılık sorumluluk üstlenen operasyonel nesnelerdir.

Seneryo – 1

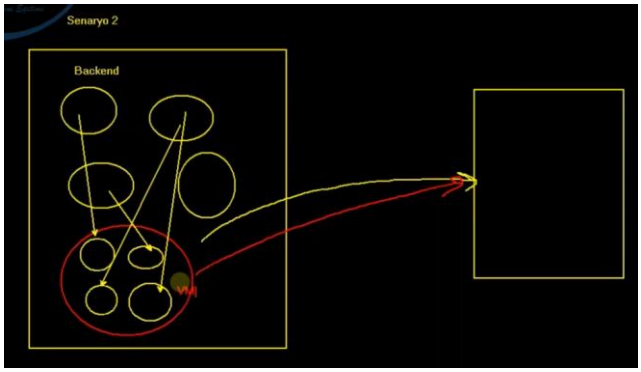
Backend'de çeşitli veriler var ve bu verilerin bazılarının View'e gönderilmesi lazım.

Tüm verileri View'e göndermektense sadece gereken verileri bir modele dönüştürüp gönderilir. Bu modele ViewModel denir.



Seneryo – 2

Backend tarafında birden fazla veri var ve bu verilerin View'e gönderilmesi gerek. Verileri tek tek göndermektense bir model oluşturulup bu model ile gönderilmesi tercih edilir.



DTO(Data Transfer Object) Nedir ?

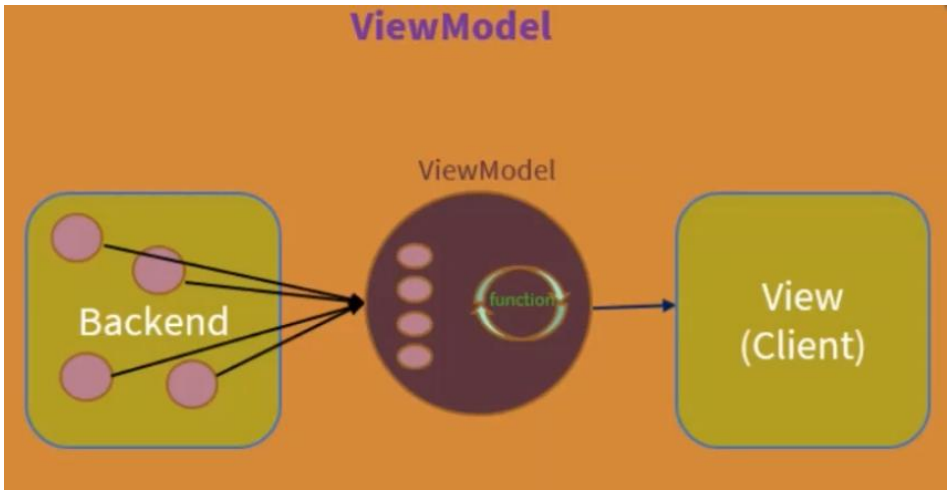
DTO, veri transferi yapılmasını sağlayan nesnedir.

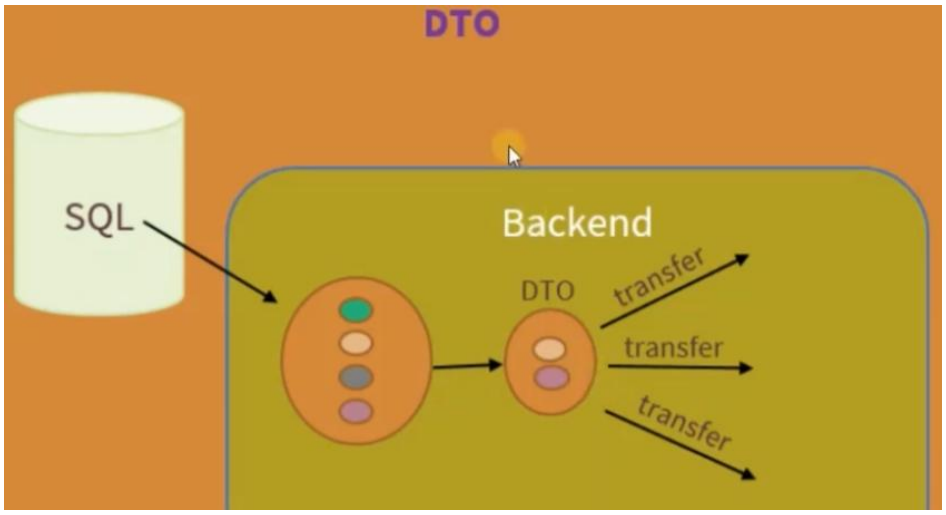
Herhangi bir davranışı yoktur. Sadece uygulama içerisinde veri tüketimi ve veri iletimi içi kullanılan, veritabanındaki herhangi bir verinin transfer nesnesidir.

DTO ile ViewModel Arasındaki Farklar

ViewModel	DTO
View'de sunulacak verinin View'e uygun ve beklenen şekilde tasarlanmış modeli	Bir verinin(genellikle veritabanından gelen verinin) transfer modellemesidir. Transfer edilecek verideki sadece ihtiyaç olan dataları temsil eder.
Veriyi sunum/presentation için anlamlı hale getirir.	Sunum için de kullanılabilir ama bunun dışında Backend içerisinde veri transferi için de kullanılır.
İçerisinde işlevsel fonksiyonlar barındırabilir.	Fonksiyon barındırmaz.
İçerisinde bir veya birden fazla DTO temsil edilebilir.	Sadece ham veri barındırır.
DTO'ya nazaran daha karmaşıktır.	

Temel Amaç


ViewModel
ViewModel
Backend
function
View
(Client)


DTO
SQL
Backend
DTO
transfer
transfer
transfer

Seneryo - 1

Personel isimli bir Entity olsun, bu Entity içerisinde çeşitli Property'ler var. Ama bir View içerisinde personelin sadece Name ve Lastname değerli geliyor. Bu durumda View'e model olarak Personel verilemez çünkü değer verilmeyen diğer Property'ler açıkta kalıyor. Bu durum için bir ViewModel oluşturulur.

Proje altında ViewModels isimli bir klasör oluşturulur. Bu isimlendirme zorunlu değil. Raconsaldır.

ViewModel oluşturulurken verilen isim yapılan operasyona uygun olmalıdır. Yani örneğin personel eklenecekse "PersonelCreateViewModel" yapılmalıdır. Çünkü ilerde Personel ile ilgili başka operasyonlarda da ViewModel kullanılabilir.

ViewModel içerisinde taşınmak istenilen verinin Property'leri tanımlanır. Taşınan Entity'nin değil o Entity'nin sadece taşınan Property'leri ViewModel içerine tanımlanır.

Burada Property isimlerinin aynı olması önerilir. Raconsaldır.

Bu tanımlamalardan sonra View içerisinde Personel değil PersonelCreateViewModel model olarak verilir.

Controller Post Action kısmında da PersonelCreateViewModel beklenir.

Aynı seneryoda bir listeleme yapılıyor mesela. Bu listelemede ilgi personellerin sadece isim ve soyisim verileri kullanılacak bu View'de bu durumda Entity içerisindeki diğer Property'ler boşu boşuna gönderilmiş oldu. Mesela bir Entity içerisinde 20 tane Property olsaydı her Entity için boşu boşuna bir sürü veri gönderilmiş olacaktı. Bu durumu önlemek niçin de ViewModel kullanılır.

Gerçek hayatta asla veri tabanından oluşan Entity'ler direkt olarak View tarafına gönderilmez. Bu verileri ViewModel'lar ile belirler formatlara dönüştürülüp o şekilde gönderilir.

Bu iş için bir tane PersonelListViewModel oluşturulur.

Seneryo - 2

Birden fazla farklı sınıf olasun. Mesela Personel, Müşleri ve Urun bu üç nesnenin aynı anda View'e gönderilmesi gerekiyor. Bunun için Tupple da kullanılabilir, ViewModel de kullanılabilir.

ViewModel kullanmak için, ViewModel klasörü altında bir sınıf oluşturulur.

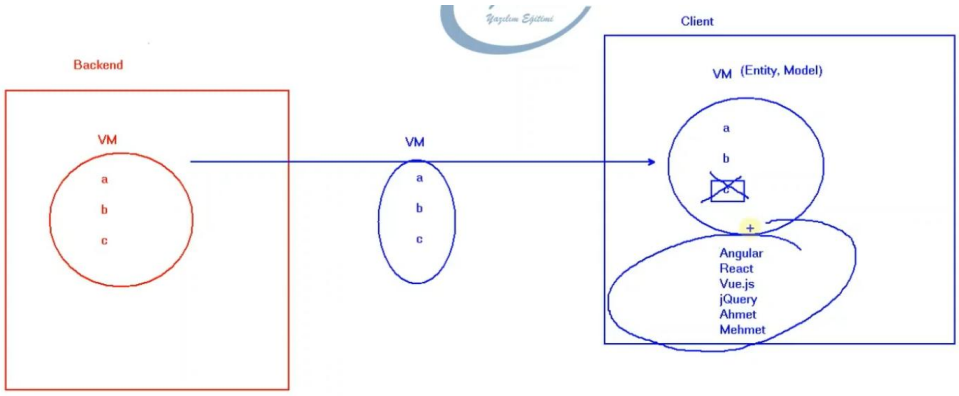
Bu sınıf içerisine kullanılacak olan sınıflar Property olarak verilir.

```
public class MergeViewModel
{
    0 references
    public Personel Personel { get; set; }
    0 references
    public Urun Urun { get; set; }
    0 references
    public Musteri Musteri { get; set; }
}
```

Sözleşme/Kontrar Mantığı Nedir ?

Backend'den ViewModel'a veri gönderiliyorsa. İlgili ViewModel'ın türü sözleşme olarak geçer.

Backend tarafında bir ViewModel oluşturuluyorsa ve Client tarafına gönderiliyorsa, Client tarafında da bu ViewModel'i karşılayaack bir Entity/Model oluşturulması gerekiyor.



ViewModel’lar da Validation Durumları

Kullanıcıdan gelen veriler Entity’ler ile değil ViewModel’lar ile karşılanmalıdır.

Alınan verilerin Validation’ları da ViewModel içerisinde yapılmalıdır.

Eğer direkt Entity üzerinden Validation yapılırsa şöyle bir senaryo olabilir. Örneğin bir Personel Entity’sinde pozisyon, medeniHal ve Adi Required olsun. Bir ViewModel’de sadece Adi ve Soyadi kısımlarını alınırsa pozisyon ve medeniHal kısımları gelmediği için mimari hata verir.

Bunun yerine ilgili Validation’lar direkt olarak ViewModel üzerinde yapılır.

```
public class PersonelCreateValidationViewModel
{
    [Required]
    0 references
    public string Name { get; set; }
    [Required]
    0 references
    public string LastName { get; set; }
}
```

ViewModel’ı Entity’e Dönüştürme

Kullanıcıdan gelen veriler ViewModel tarafından kaşılandıktan sonra bu ViewModel’da ki verileri veri tabanına kayıt etmek için ilgili verileri Entity’e dönüştürmek gerekir. Bunun için birkaç yol vardır:

- Manuel Dönüştürme
- Implicit Operator Overload ile Dönüştürme
- Explicit Operator Overload ile Dönüştürme
- Reflection ile Dönüştürme
- AutoMapper Kütüphanesi ile Dönüştürme

Manuel Dönüştürme

Tek tek elle yapılan yöntemdir. Action parametresinde gelen ViewModel’in Propertylerini tek tek ilgili nesneye karşılık gelen Property’lerine eşitlemektir.

```
[HttpPost]
0 references
public IActionResult ManuelDonusturme(PersonelCreateViewModel model)
{
    Personel p = new Personel()
    {
        Name = model.Name,
        LastName = model.LastName
    };
    // Save Database
    return View();
}
```

Implicit Operator Overload ile Dönüştürme

İleri düzey p rogramlama bilgisi gerekli. Entity sınıfı veya ViewModel sınıfı üzerinde uygulanabilir.

```
public static implicit operator PersonelCreateViewModel(Personel model)
{
    return new PersonelCreateViewModel
    {
        Name = model.Name,
        LastName = model.LastName
    };
}
```

Action tarafında ise referans vermek yeterli oluyor:

```
[HttpPost]
0 references
public IActionResult ImplicitDonusturme(PersonelCreateViewModel model)
{
    Personel personel = model;
    // Save Database
    return View();
}
```

Explicit Operator Overload ile Dönüştürme

Implicit'den farklı bilinçli tür dönüşümü gibi düşünülebilir.

Class bazında kullanımı Implicit ile birebir aynı.

Action tarafında ise dönüştürülmek istenen türe bilinçli olarak Cast edilmesi gerekiyor.

```
[HttpPost]
0 references
public IActionResult ExplicitDonusturme(PersonelCreateViewModel model)
{
    Personel personel = (Personel)model;
    PersonelCreateViewModel vm = (PersonelCreateViewModel)personel; // Ters dönüşüm
    // Save Database
    return View();
}
```

Reflection ile Dönüştürme

Bu yöntem de ileri düzey bir teknik olarak geçer.

Hem ViewModel'dan Entity'e hem de Entity'den ViewModel'a dönüşüm yapıyor aynı kodları bir daha yazmamak için Generic olarak bu işlem yapılabilir.

```
public static class TypeConversion<T, T2>
{
    0 references
    public static TResult Conversion<T, TResult>(T model) where TResult : class, new()
    {
        TResult result = new TResult();
        typeof(T).GetProperties().ToList().ForEach(p =>
        {
            PropertyInfo property = typeof(TResult).GetProperty(p.Name);
            property.SetValue(result, p.GetValue(model));
        });

        return result;
    }
}
```

İleri düzey ve zor bir teknik.

AutoMapper Kütüphanesi ile Dönüştürme

Genellike projelerde kullanılan bir kütüphanedir.

Nuget ile AutoMapper kütüphanesi indirilmesi gerek.

Proje klasörü altında bir AutoMappers klasörü oluşturulur ve bu klasör altında hangi Entity ile çalışıyorsa onun için bir profil Class'I oluşturulur. Mesela Personel için "PersonelProfil" isimli bir Class oluşturulur. Bu Class konfigürasyon görevi görür.

Bu PersonelProfil sınıfının Profile Class'ından türemesi gerekiyor.

PersonelProfil sınıfının Consutractoru içerisinde dönüştürülmek istenen sınıflar CreateMap fonksiyonu ile dönüştürülür.

```
public class PersonelProfil : Profile
{
    0 references
    public PersonelProfil()
    {
        CreateMap<Personel, PersonelCreateViewModel>();
        CreateMap<PersonelCreateViewModel, Personel>(); // Ters Dönüşüm
    }
}
```

Bu kütüphaneyi kullanabilmek için Program.cs dosyası içerisinde servis olarak eklemek gerekiyor. Fakat bu kütüphaneyi Dependecy Injection ile kullanabilmek için “AutoMapper.Extensions.Microsoft.DependencyInjection” kütüphanesini de projeye eklemek gerekiyor.

Kütüphaneyi ekledikten sonra AddAutoMapper fonksiyonu ile Container’a eklenir.

AddAutoMapper fonksiyonuna parametre olarak typeof ile ilgili dönüşüm sınıfı verilir. Mesela PersonelProfil

```
builder.Services.AddAutoMapper(typeof(PersonelProfil));
```

Kullanılmak istenen Controller’a Dependency Injection ile Context olarak IMapper sınıfı çekilir.

Action içerisinde bu _mapper field’ı ile istenilen dönüşüm yapılabilir.

_mapper üzerinden Map() fonksiyonu ile dönüşüm yapılır. Map fonksiyonuna Generic olarak dönüştürülmek istenen Class verilir, parametre olarak ise dönüştürülecek model verilir. Geriye istenilen türden Class döner.

```
[HttpPost]
0 references
public IActionResult AutoMapperDonusturme(PersonelCreateViewModel model)
{

    Personel p = _mapper.Map<Personel>(model);
    PersonelCreateViewModel vm = _mapper.Map<PersonelCreateViewModel>(p); // Ters Dönüşüm

    // Save Database
    return View();
}
```